

Jxiv

[ジェイカイク / dʒéikaiv]

Title	Study and evaluation of quantum computer automatic offloading of specific calculation processing for environmental adaptation
Author(s)	Yoji Yamato
Citation	Journal title(Repository name etc.), Volume, Issue, Pages(Article number) etc. ・ ジャーナル名 (刊行物・サイト名) ・ 巻号 ・ ページ (その他論文番号等) : International Journal of Parallel, Emergent and Distributed Systems ・ DOI (URL) https://doi.org/10.1080/17445760.2025.2450669 Publication Date: yyyy/mm/dd ・ 出版日 : 2025 年 1 月 27 日 Publisher ・ 出版者 : Taylor and Francis
Declaration	This preprint is the Author Accepted Manuscript(AAM) of the above. ・ 本プレプリントは、上記論文の (受理済)著者最終稿(AAM) である。 All necessary permissions from the publisher have ・ ジャーナル (出版者) から必要な許諾を been obtained not been obtained ☒ 得ている ☐ 得ていない
Notes	"This is an Accepted Manuscript of an article published by Taylor & Francis in International Journal of Parallel, Emergent and Distributed Systems, on 27th Jan., 2025, available online: https://doi.org/10.1080/17445760.2025.2450669 ."

ORIGINAL PAPER

Study and evaluation of quantum computer automatic offloading of specific calculation processing for environmental adaptation

Yoji Yamato^a

^aNTT Corporation, 3-9-11 Midori-cho, Musashino-shi, Tokyo 180-8585, Japan

ARTICLE HISTORY

Compiled October 4, 2024

ABSTRACT

In recent years, applications using heterogeneous hardware such as GPUs and FPGAs are on the rise. However, to fully utilize these applications, an understanding of the hardware is necessary, and the barrier to success is high. In light of this, we have proposed an environment-adaptive software that automatically converts code written by programmers for regular CPUs according to the environment in which it is deployed, enabling high-performance operation. This paper focuses on quantum computers in particular as a part of automatic offloading to new hardware. We achieve automatic offloading by analyzing existing applications to be offloaded, discovering specific computational processes such as the eigenvalue solvers that are suitable for quantum computers, and replacing those parts with processes that use quantum computers. We confirm that the proposed method can be automatically offloaded to a quantum computer and executed by measuring the actual processing time on Microsoft Azure Quantum.

KEYWORDS

Environment Adaptive Software; Automatic Offloading; Quantum Computer; Qiskit; Similarity Detection

1. Introduction

In recent years, the slowdown of Moore's Law, which was expected to lead to faster CPUs, has been predicted. As a result, the use of not only CPUs but also heterogeneous hardware such as GPUs (Graphics Processing Units) and FPGAs (Field Programmable Gate Arrays) in systems is increasing. For example, Amazon provides GPU and FPGA servers as cloud (e.g., [1]-[4]) instances [5]. Microsoft is working on using FPGAs to improve the efficiency of Bing searches [6]. In addition to GPUs and FPGAs, the number of IoT devices is also increasing (e.g., [7]-[11]), and new hardware such as quantum computers and DPUs (Data Processing Units) are also emerging.

However, in order to speed up a system using hardware other than normal cores CPUs, it is essential to develop and configure programs that take into account the characteristics of that hardware, and this requires programming skills such as C language extensions such as OpenMP (Open Multi-Processing) [12], CUDA (Compute Unified Device Architecture) [13], and OpenCL (Open Computing Language) [14], which represents a high hurdle for most programmers who are primarily used to developing in scripting languages such as Python and Javascript.

Systems that use heterogeneous hardware other than CPUs with normal number of cores, such as GPUs, FPGAs, quantum computers, and DPUs, are expected to become more common in the future, with the expectation of reducing power consumption, but there is a high skill barrier to implementing them. Therefore, in order to remove the skill barrier and make full use of hardware other than CPUs with normal number of cores, a platform is becoming important that allows programmers to convert, configure, and adapt software written with the same processing logic as normal programs to the environment in which it will be deployed (GPU, FPGA, quantum computer, etc.).

Therefore, we have proposed environment-adaptive software that automatically converts existing software code for GPUs and FPGAs and configures resources so that the code can utilize the GPUs and FPGAs of the target environment, thereby making the application run at high speed. Additionally, as elements of the environment-adaptive software, we have proposed and evaluated a method for automatically offloading loop statements of existing software code to GPUs and FPGAs, and a method for reconfiguring processes that have been offloaded once in response to changes in the situation during operation [15]-[22].

However, our verification so far has mainly focused on conversion, configuration, and reconfiguration using existing accelerators such as GPUs and FPGAs, and automatic offloading to new hardware such as quantum computers and DPUs has not been considered. For example, quantum computers are currently provided by many companies, including D-wave, Amazon, Microsoft and IBM, and are being applied to NP (Non-deterministic Polynomial) problems. Some companies, such as Amazon and Microsoft, rent quantum computers by the hour on their clouds, so users can be used without huge initial costs. DPUs are new hardware that accelerate specific data processing, and although still limited, they are beginning to be used.

This paper focuses on quantum computers in particular as a part of automatic offloading to new hardware. We aim to develop a general-purpose method that can be applied to new hardware such as DPUs that accelerate specific processing. First, we analyze the existing application to be offloaded, find specific calculation processes such as the eigenvalue analysis problem that are suitable for quantum computers, and replace that part with processing that uses a quantum computer to achieve automatic offloading. Quantum computer algorithms, such as how to solve the eigenvalue analysis problem on a quantum computer, are outside the scope of this paper; our target is to find and replace problems such as the eigenvalue analysis problem that are suitable for quantum computers. We confirm that the proposed method can be automatically offloaded to a quantum computer and executed by measuring the actual processing time on Microsoft Azure Quantum [23].

This paper is structured as follows: Section 2 provides an overview of existing technologies and explains the environment-adaptive software previously proposed by us. Section 3 analyzes existing applications and proposes a specific method for offloading to a quantum computer. Section 4 explains the implementation tools and an overview of the implementation operation of the proposed method. Section 5 confirms the proposed method through Azure Quantum offloading of existing applications. Section 6 compares with related work. Section 7 concludes.

2. Existing technologies

2.1. Market technologies

NVIDIA provides CUDA as an environment for GPGPU (General Purpose GPU) (e.g., [24]), which uses the computing power of the GPU for purposes other than image processing. CUDA targets only GPUs, but there is OpenCL, which has specifications for handling heterogeneous hardware such as GPUs, FPGAs, and multi-core CPUs in a similar manner, and various companies have released development environments for it. CUDA and OpenCL are C language extension programs, and are highly difficult (such as writing code to release and copy memory data between the GPU kernel and the CPU host).

Compared to CUDA and OpenCL, there are technologies which make it easier to use heterogeneous hardware by using directives to specify the parts to be processed in parallel, and the compiler creates binary files suited to GPUs, multi-core CPUs, etc. according to the directive commands. Specifications include OpenMP and OpenACC [25], and compilers include gcc and the PGI compiler [26].

By using technologies such as CUDA, OpenCL, OpenMP and OpenACC, it is now possible to offload to GPUs, FPGAs and multi-core CPUs. However, even if the processing itself can be done, there are challenges in accelerating it. For example, there is the Intel compiler [27], which is an automatic parallelization for multi-core CPUs. When parallelizing, these parallelize parts of a loop statement that can be processed in parallel. However, simply parallelizing a loop statement does not provide sufficient performance due to the effects of memory processing, etc. When accelerating on GPUs or FPGAs, CUDA or OpenCL experts repeatedly tune the system, or use OpenACC compilers to search for suitable parallel processing parts. For this reason, it is difficult for programmers with low skills to accelerate on heterogeneous hardware, and even if automatic parallelization technologies are used, it is necessary to carry out trial and error on the parallel processing parts. As an automated trial and error approach, we propose automatic offloading using evolutionary computing.

A quantum computer is a computer that applies the principles of quantum mechanics to calculations. D-wave and other systems use the quantum annealing method, but the development of quantum computers using the more versatile quantum gate method is becoming mainstream, and many companies such as Amazon, Microsoft, Google and IBM are working on this. AWS Braket and Azure Quantum are provided on Amazon's AWS cloud and Microsoft's Azure cloud, respectively, and can be used on an hourly basis as clouds, with no initial hardware costs for the quantum computer itself. However, in reality, quantum algorithms for solving NP problems such as the traveling salesman problem, the eigenvalue analysis and prime factorization using quantum computers are researched and devised one by one by experts, making them difficult for amateurs to use.

A DPU is a programmable computer processor that tightly integrates a general-purpose CPU and network interface and hardware; NVIDIA's BlueField [28] is a well-known example. It is used in place of a conventional NIC (Network Interface Card) to offload complex network processing and other infrastructure processing from the main CPU.

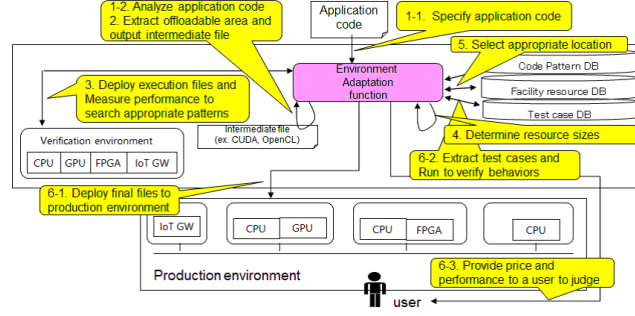


Figure 1. Processing steps of environment-adaptive software

2.2. Outline of environment-adaptive software

As the environment-adaptive software, we propose the processing steps shown in Figure 1. The environment-adaptive software has an environment adaptation function provided by the operator at its core, and operates by linking modules of the verification environment, production environment, facility resource DB, test case DB, and code pattern DB.

- Step 1: Analysis of user-provided code:
- Step 2: Extraction of offloadable parts:
- Step 3: Search for appropriate offloadable parts:
- Step 4: Adjustment of production resource amount:
- Step 5: Adjustment of production deployment location:
- Step 6: Production deployment of binary files and verification:
- Step 7: Reconfiguration during actual operation:

First, the code provided by the user is analyzed, and in Steps 1-6, before actual operation begins, the code is converted, the amount of resources is adjusted, the placement location is adjusted, and verification is performed through testing in a verification environment. In Step 7, after actual operation begins, practical data is analyzed, and reconfiguration is performed if changing the configuration would be more cost-effective.

2.3. Problems in this paper

Now we summarize the problems in this paper. High-speed processing on heterogeneous hardware is mainly done manually by experts. We have proposed environment-adaptive software and have also realized an automatic offloading method for GPUs and FPGAs, but offloading to new hardware such as quantum computers or DPUs has not been considered. Therefore, this paper focuses on quantum computers offloading. Although we will not verify DPUs, we will perform general-purpose offloading that can also be applied to new hardware such as DPUs that offload specific processing. The content to be offloaded will be processing logic for existing normal CPUs provided by the user. There are no examples in the market of non-expert users using quantum computers, and as both the difficulty and the benefits are great, we will first tackle quantum computers.

3. Processing offload to a quantum computer

To realize the concept of environment-adaptive software, we have proposed automatic offloading of loop statements to GPUs and FPGAs, automatic adjustment of resource amounts and placement, and reconfiguration during actual operation. Based on these elemental technologies considerations, in 3.1 we describe quantum computer characteristics and basic considerations based on them. In 3.2 we describe the discovery of the offload parts necessary for offloading based on quantum computer characteristics. In 3.3 we describe the use of the discovered offload parts on quantum computers.

3.1. Quantum computer characteristics and basic considerations

As mentioned in Section 2, many companies are currently involved in the development of quantum computers using quantum gate methods, and the number of quantum bits being offered is increasing. In order for quantum computers to exhibit quantum effects, it is almost essential to keep them at low temperatures using a dilution refrigerator, and purchasing one would cost over one million USD. In order to use a quantum computer without the initial costs, it is possible to use quantum computers provided as cloud services such as Amazon Braket and Azure Quantum. For example, performance computing on Azure Quantum costs about 90 USD per hour. Therefore, these cloud-based quantum computers are used, but even though there is no initial cost, the hourly price is higher than that of a VM equipped with a normal CPU or GPU. GPUs and FPGAs use evolutionary computing techniques that find increasingly faster patterns through repeated testing in a verification environment. Since it is cost-prohibitive to occupy a quantum computer for long periods of time for verification, when offloading to a quantum computer, it is necessary to offload only effective calculations in one go, without repeated testing.

Now we consider offloading calculations that are effective on a quantum computer. Quantum computers using the quantum gate method are said to be able to solve problems with a huge number of combinations, such as the traveling salesman problem, the eigenvalue analysis problem, and prime factorization, much faster than normal CPUs, and experts who understand quantum mechanics design quantum algorithms suitable for each problem to solve them.

In this paper, we therefore identify the calculation parts that would be effective on a quantum computer in the code that the user requests to offload, and automatically offload these calculation parts by using a cloud quantum computer according to the quantum algorithms devised by experts.

3.2. Searching the suitable computational part for offloading

Following the basic considerations in the previous subsection, the discovery of computational parts suitable for quantum computers is explained with reference to Figure 2.

A. Discovery of computational parts.

The code provided by the user is parsed, and computational parts such as classes and structures are detected from the source code. The type of problem, such as the traveling salesman problem, the eigenvalue analysis problem, cannot be determined without actually looking at the contents, but the computational parts are detected first.

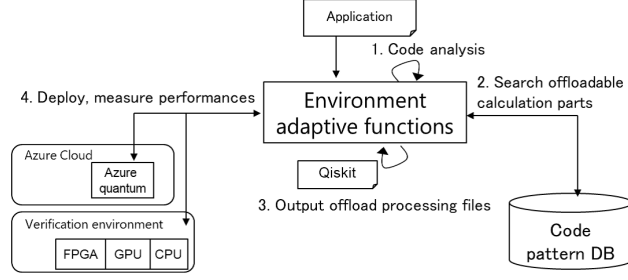


Figure 2. Offload method to quantum computers

B. Searching a computational part suitable for a quantum computer

First, codes for processing the traveling salesman problem, the eigenvalue analysis problem, prime factorization and so on are stored in the code pattern DB as computational parts suitable for a quantum computer. Furthermore, the processing parts (equivalent to libraries) for using a quantum computer with the quantum algorithm corresponding to that code are also stored in the code pattern DB.

The calculation parts suitable for a quantum computer are detected by comparing them with information registered in the code pattern DB. A similarity detection tool is used to detect whether there are any quantum algorithms that can be offloaded for the calculation parts of classes, structures and so on detected in A step. Similarity detection tools are tools such as Deckard [29] that make it possible to detect similar code such as code that has been copied and then modified. Similarity detection uses similarities in abstract syntax trees, lexical units, lines, program dependency graphs, fingerprints and so on, while Deckard uses abstract syntax trees to analyze C/C++ and Java language code. Of course, it is likely that there will be many cases where a calculation part suitable for a quantum computer cannot be found, but in such cases, it is sufficient to move on to trials of accelerating the code using evolutionary computing methods in a verification environment on GPUs, FPGAs and on other hardware.

The tool's thresholds are used to determine whether the calculation parts suitable for quantum computers registered in the code pattern DB are highly similar. Since it is clear that similarity detection cannot cover 100% of the range, the application of deep learning recognition for AI (Artificial Intelligence) processing is also considered as a future application. However, even in the case of AI processing, it is difficult to make a machine understand the intent of the code, so classes, structures and so on that are newly developed independently are not considered to be eligible for quantum computer offloading.

When a calculation part suitable for a quantum computer is discovered, it is offloaded to the quantum computer using the processing part (equivalent to a library) for using a quantum computer with the quantum algorithm corresponding to that code registered in the code pattern DB.

3.3. Quantum computer processing

The calculation parts of classes, structures and so on are searched for whether the calculations are suitable for a quantum computer, and if so, the processing part (equivalent to a library) is replaced on the host side and implemented. Since quantum computers differ from one development company to another, the processing is generally

implemented using tools provided by each company, but there are also frameworks that can be used by plural companies. Qiskit [30] is a quantum computer framework open sourced by IBM, Cirq is a quantum computer framework open sourced by Google, and Q# is a programming language for quantum computers proposed by Microsoft. For example, when a user application containing the traveling salesman problem is specified, a quantum algorithm implemented in Qiskit is discovered from the code pattern DB using Deckard, and it is used for processing in Azure Quantum.

Here, a part (equivalent to a library) that is processed by a quantum computer for a specific calculation part is found, so it becomes necessary to generate an interface part. Since the calculation is judged to be suitable for quantum computing based on similarity, there is no guarantee that the number and type of arguments and return values expected by the implementation part such as Qiskit and the host program match. If they do not match, since quantum algorithm implementation parts cannot be changed frequently, the user who requests offloading is asked whether they will change the number and type of arguments and return values in the original code to match the quantum computer processing part, and after confirmation and approval, an offload confirmation test is attempted. Regarding the difference in type, if it is only necessary to cast between float and double or so on, it is possible to start the confirmation test without particular user confirmation. Also, if the number of arguments or return values differs between the original program and the quantum computer processing part, for example, arguments 1 and 2 are required and 3 is optional in the CPU program, and arguments 1 and 2 are required in the quantum computer processing part, and there is no problem if they are omitted, optional arguments may be automatically treated as absent without confirmation from the user.

In this way, similarity detection technology is used to detect the computational parts suitable for a quantum computer from the code requested by the user to be offloaded, and automatic offloading to a quantum computer is achieved by replacing them with quantum computer processing parts implemented in an open source framework.

4. Implementation

4.1. Tools to use

We explain the implementation to confirm the effectiveness of the proposed method. To confirm the effectiveness of quantum computer offloading, the target applications are C/C++ and Python applications, which are numerous.

For C/C++ language syntax analysis, we use the LLVM/Clang 6.0 syntax analysis library (libClang python binding) [31]. For Python syntax analysis, we use Lark [32].

For C/C++ language, we use Deckard v2.0 [29] as a similarity detection tool. In order to apply quantum computer offloading to specific calculation parts, Deckard compares the calculation parts that have been modified from existing code, and determines the similarity of the abstract syntax trees of code registered in a DB using a threshold. For Python, we use CloneDigger [33].

The code pattern DB used for matching uses MySQL8. Along with the calculation code used for matching, the Qiskit framework implementation of the quantum computer processing part associated with it is also stored.

The quantum computer framework uses Qiskit [30], open sourced by IBM. Qiskit SDK v1.1.2 is used.

The quantum computer uses Azure Quantum [23], a quantum gate method, as a

function of the Microsoft Azure cloud service. As mentioned earlier, even hourly use of the cloud is quite expensive, making it difficult for users to use it repeatedly for verification, and therefore evolutionary computing methods cannot be used.

The implementation will be done in Python 3.

4.2. Implementation behavior

The implementation involves the following processes.

As a preliminary step, the matching code and the part that utilizes a quantum computer are registered in a MySQL code pattern DB. The matching code is C/C++ or Python code that describes the traveling salesman problem, eigenvalue analysis problem and so on. The part that utilizes a quantum computer is registered as the processing part implemented in Qiskit with the corresponding quantum algorithm. It is expected that this matching code and the part that utilizes a quantum computer will be prepared in advance by cloud service providers and other such providers.

First, the application code that the user requests to offload is received. The application is written in C/C++ or Python. Depending on the description language, parsing is performed using Clang for C/C++, or Lark for Python. Parsing identifies classes, structures and so on, which are the units of the computational part.

Deckard then detects whether there are any quantum algorithms that can be offloaded from the detected computational parts such as classes and structures. Since differences in parameter values do not affect the abstract syntax tree, a search is performed with a similarity threshold of 95% for the abstract syntax tree. If nothing is found here, our previous technology [22] can be used to attempt to speed up the process by offloading to a GPU or FPGA.

If a calculation part suitable for a quantum computer is discovered, the processing part registered in the code pattern DB and implemented with the associated Qiskit is replaced with the relevant part of the user program, and then implemented. Because the quantum computer used in this case has been decided to be Azure Quantum, in addition to the Qiskit processing part, the address information for Azure Quantum is also obtained from the code pattern DB and set in the user program.

If there is a mismatch between the implementation part of Qiskit and the host program in terms of the number or type of arguments or return values, the user is asked whether they would like to change the number and type of arguments and return values in the original code to match the quantum computer processing part. If the difference in type can be easily resolved by simply casting between float and double, for example, there is no need to confirm with the user.

Once the parameters have been set, a confirmation test is performed to offload the processing to the quantum computer, and the overall processing time and the processing time on the quantum computer are measured. The measurement results and the usage fee (hourly price for the host VM + hourly price for the quantum computer of Azure Quantum) are presented to the user, and if the user agrees, the actual service begins.

5. Evaluation

The proposed method enables automatic offloading of processing from user-specified applications to a quantum computer. We evaluate the effectiveness of the proposed method by confirming that the quantum computer processing time is appropriate.

5.1. Evaluation conditions

5.1.1. Evaluation subject

The subject of evaluation is an eigenvalue analysis problem. For high-dimensional complex matrices, there is no calculation procedure that can accurately express the eigenvalues using a finite number of algebraic operations. For this reason, iterative methods have traditionally been used for the numerical analysis of eigenvalue problems. This can be accelerated by utilizing the quantum state superposition properties of quantum computers. The essence of the problem is to find the eigenvalues of the associated matrix of a general n -th degree algebraic equation, and each problem depends on differences in parameters. The eigenvalue analysis problem used is [34]. As [34] shows, solutions to eigenvalue analysis problems on quantum computers are also implemented in Qiskit using quantum algorithms.

Of course, quantum computers are being researched to use quantum algorithms to solve NP problems such as the knapsack problem and prime factorization, but one of the offloading subjects for quantum computers is the eigenvalue solver which has a wide range of applications for general users.

5.1.2. Evaluation method

Assuming one user, the user requests the offloading of an application that includes the eigenvalue solver. When the operator's platform receives the request, it analyzes the application, searches for the offloaded calculation part using a similarity detection tool, and if found, replaces it with the part that uses a quantum computer and offloads it to the quantum computer. The processing time during the offloaded process is measured, and the measurement results and the usage price are presented to the user.

The experimental conditions are as follows:

Offloaded calculation part: Eigenvalue Solver [34]

Offload destination: Azure Quantum

Method of searching offloaded calculation part: The calculation part in the application code is compared with the code in the code pattern DB using the similarity detection tool Deckard.

Performance measurement: The processing time on a quantum computer is measured using the sample parameters in [34]. At small sizes, the calculation can be completed in a reasonable time even with a normal CPU, but this time we will not measure at large sizes in order to confirm quantum computer offloading.

5.1.3. Evaluation environment

The evaluation environment and specifications are shown in Figure 3. Here, the user of the note PC using the verification machine specifies the application they want to offload, and the verification machine uses Deckard to search and compare it with a DB on the same node. When offloading, the Qiskit implementation for quantum computers is obtained and the processing is requested to Azure Quantum on the Azure cloud via the Internet. The offload processing and overall processing time are measured by the verification machine.

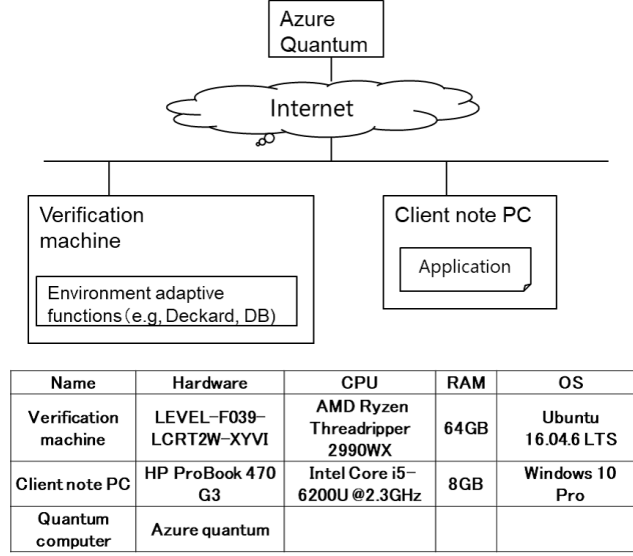


Figure 3. Evaluation environment

Application	Quantum computer	Processing time
Eigenvalue Solver	Azure Quantum	15.7 sec

Figure 4. Processing time of quantum computer for eigenvalue solver

5.2. Result and discussions

Figure 4 shows the processing time when offloading to Azure Quantum using the proposed method. The work was offloaded to Azure Quantum as expected from the request. Due to the parallelism of quantum computers, it is expected that the eigenvalue solver will be completed in a short time even if the size of the problem increases, and in this case it was completed in 15.7 seconds. The time to offload itself is completed in seconds, from the user request to the Deckard search, replacement of the Qiskit implementation and so on, so there is no particular waiting time for the user. Of course, because Azure is used as an external service, when the user themselves, rather than the service provider, signs a contract for Azure, screening is required, so a certain amount of time and operation is required before the contract can be signed.

Previous researches into offloading loop statements to GPUs and FPGAs involved measuring the performance of multiple offloading patterns in a verification environment and selecting the fastest pattern; for example, even a large-scale application such as Darknet, which has over 100 for statements, was automatically offloaded to a GPU and achieved a three-fold increase in speed. However, there are many examples where performance has been improved by more than ten times by manual development using CUDA for GPUs and HDL for FPGAs, and there have been cases where the degree of performance improvement was insufficient. The method proposed this time offloads the calculation parts that are suitable for quantum computers to a quantum computer using Qiskit, making it possible to process problems that cannot be solved by existing

CPUs when their size becomes large, such as the eigenvalue solver.

However, the proposed method is based on the assumption that the processing parts of Qiskit that have already been implemented and examined by experts have been registered in a DB, and therefore it is not possible to offload the parts of the calculations that are not suitable for quantum computers and are not registered in the DB. Therefore, we believe that in practical operation it will be necessary to combine this with methods such as GPU or FPGA offloading of loop statements, which were previously studied.

Now, we consider cost performance. There are still few companies that offer quantum computers on the cloud, but for example, Azure Quantum can be used for about 90 USD per hour. As a result, using it for a full month would cost around 65,000 USD, making it unrealistic. A more realistic use would be for users to use it for just one hour to calculate a solution to determine. On the other hand, if the problem size is small, a solution can be obtained in a reasonable amount of time even with a CPU VM, so it can be said that cost-effectiveness becomes apparent in use cases where a problem of a size that can only be solved by a quantum computer is used for just several hours.

Regarding the time it takes to start using the service, in the case of quantum computer offloading, it is expected that processing will be completed in a matter of seconds. If there is no calculation part that can be offloaded, it would take experts several months to study a quantum algorithm, so we speed up the loop statements using GPU or FPGA offloading techniques, and accelerate the speed within a verification period of a few hours to a day. When providing our service, we plan to have users try it out for the first day, during which we will try out speeding it up in a verification environment, and from the second day we will be able to provide a production service using GPUs, FPGAs, quantum computers and so on, so we believe that taking one day until the actual service starts is within an acceptable range.

In order to offload to a quantum computer using the proposed method, it is necessary to pre-register the calculation parts commonly used in many applications in a DB such as the Qiskit implementation. Quantum computers themselves are said to be suitable for problems such as the traveling salesman problem, the eigenvalue analysis problem, and prime factorization, which take advantage of the parallelism of quantum mechanics, and it is expected that they will be registered specializing in specific calculation domains.

In addition, while parameter differences and slightly modified code can be discovered by similarity detection of abstract syntax trees, we will examine in more detail the extent to which this can be discovered. In addition to similarity detection, it is also necessary to consider whether deep learning processing in the field of AI can be used to more widely discover calculation parts that can be offloaded to quantum computers.

6. Related work

Works on offloading to GPUs include [35], [36] and [37]. The work of [35] mentions the use of metaprogramming and JIT compilation to offload C++ expression templates to the GPU, while [36][37] are working on offloading to the GPU using OpenMP. There is little research into automatically converting existing code to be GPU-compatible, as we aim, without using a new development model or manually inserting directives.

There have been many studies on FPGA offloading [38][39][40]. The work of Liu [38] proposed a method to offload nested loops to FPGAs and further showed that nested loops can be offloaded with 20 minutes of manual work. These methods require

manual addition of instructions such as parts to be parallelized using OpenMP or other specifications. Therefore, there is little research on automatically offloading or reconfiguring existing code to FPGAs, as in this paper.

Many companies, including Amazon, Microsoft, Google, IBM and D-wave are working on quantum computers. Other researchers using Azure Quantum for calculation processing, similar to the method used in this paper, include [41] and [42]. These researchers use development kits and tools to have Azure Quantum perform the calculations. However, the quantum algorithms themselves need to be designed by experts who understand quantum mechanics, and are not intended to be used by amateurs, as in this paper. Azure Quantum can use Q# proposed by Microsoft, but it can also use Qiskit and Cirq, which has been open-sourced by IBM and Google. The Qiskit used in this paper can be used with quantum computers other than Azure Quantum.

DPU is a new type of hardware that has recently emerged, and [43] is considering offloading NIC processing. Since it is suitable for offloading specific processing such as NICs, it is expected that the number of providers will increase in the future. However, since knowledge of hardware is required, efforts such as this paper that can be used without specialized knowledge are desirable.

In software engineering, there are efforts to search for code clones and make changes more efficient during the software maintenance phase by using similarities in abstract syntax trees, lexical units, lines, program dependency graphs, fingerprints and so on. These efforts are aimed at maintaining software, and do not consider offloading specific computational processes to new hardware such as quantum computers.

As described above, there are increasing efforts to speed up applications by offloading processing to heterogeneous hardware. However, most of these involve manually adding instructions of OpenMP or etc to specify which parts to parallelize, and then offloading according to those instructions; there are almost no efforts to automatically offload existing code. In particular, with regard to the use of quantum computers, frameworks that can be used on quantum computers from multiple companies have emerged, but these require consideration of quantum algorithms and are not easily usable by general users. In this paper, we analyze user applications, find the parts of calculations that are suitable for quantum computers, and replace them with Qiskit, making it possible for general users to use quantum computers.

7. Conclusion

In this paper, we propose a new element of environment-adaptive software: a method to analyze the processing logic of applications provided by users and automatically offload the appropriate parts of processing to a new type of hardware, a quantum computer.

Quantum computers are said to be suitable for huge combination problems such as the traveling salesman problem due to the superposition properties of quantum mechanics. Therefore, when a user specifies an application to be offloaded, a search is made to see if there is a computational process that is suitable for offloading to a quantum computer. The search is performed using the similarity of abstract syntax trees, specifically Deckard. The database collated by Deckard contains computational process codes suitable for quantum computers and the corresponding Qiskit modules for quantum computer processing. When a computational process suitable for a quantum computer is found based on the similarity of the abstract syntax trees, the quantum computer is used by replacing it with the corresponding Qiskit module. In this ver-

ification, the computational process to be offloaded was set to the eigenvalue solver, and the offload destination was set to Azure Quantum. The offloaded computational process was actually found using Deckard, replaced with Qiskit, and the process was offloaded to Azure Quantum. The process was completed in 15.7 seconds, confirming the effectiveness of the method.

We have two plans for the future. One is to apply the method of finding and offloading specific processes not only to quantum computers but also to new hardware such as DPUs. The other is that, now that offloading to quantum computers is possible, we will consider optimally offloading user application processing logic to the right place in a mixed environment that includes quantum computers in addition to GPUs, FPGAs, and multi-core CPUs.

Disclosure statement

The author declares no conflicts of interest associated with this manuscript.

Notes on contributors

Yoji Yamato received a B.S. and M.S. in physics, and a Ph.D. in general systems studies from the University of Tokyo in 2000, 2002, and 2009. He joined NTT in 2002, where he has been conducting developmental research on a cloud computing platform, an IoT platform and a technology of environment adaptive software. Currently, he is a distinguished researcher of NTT Network Service Systems Laboratories. Dr. Yamato is a fellow of IEICE, a senior member of IEEE and a senior member of IPSJ.

References

- [1] O. Sefraoui, M. Aissaoui and M. Eleuldj, "OpenStack: toward an open-source solution for cloud computing," *International Journal of Computer Applications*, Vol.55, No.3, 2012.
- [2] Y. Yamato, "Automatic Verification for Plural Virtual Machines Patches," *The 7th International Conference on Ubiquitous and Future Networks (ICUFN 2015)*, pp.837-838, July 2015.
- [3] Y. Yamato, "Use Case Study of HDD-SSD Hybrid Storage, Distributed Storage and HDD Storage on OpenStack," *19th International Database Engineering & Applications Symposium (IDEAS'15)*, pp.228-229, July 2015.
- [4] Y. Yamato, Y. Nishizawa and S. Nagao, "Fast Restoration Method of Virtual Resources on OpenStack," *IEEE Consumer Communications and Networking Conference (CCNC 2015)*, pp.607-608, Jan. 2015.
- [5] AWS EC2 web site, <https://aws.amazon.com/ec2/instance-types/>
- [6] A. Putnam, A. M. Caulfield, E. S. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmailzadeh, J. Fowers, G. P. Gopal, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J.-Y. Kim, S. Lanka, J. Larus, E. Peterson, S. Pope, A. Smith, J. Thong, P. Y. Xiao and D. Burger, "A reconfigurable fabric for accelerating large-scale datacenter services," *Proceedings of the 41th Annual International Symposium on Computer Architecture (ISCA'14)*, pp.13-24, June 2014.
- [7] M. Hermann, T. Pentek and B. Otto, "Design Principles for Industrie 4.0 Scenarios," *Rechnische Universitat Dortmund*. 2015.
- [8] Y. Yamato, Y. Fukumoto and H. Kumazaki, "Proposal of Real Time Predictive Maintenance,"

- nance Platform with 3D Printer for Business Vehicles," *International Journal of Information and Electronics Engineering*, Vol.6, No.5, pp.289-293, 2016.
- [9] H. Noguchi, T. Demizu, M. Kataoka and Y. Yamato, "Distributed Search Architecture for Object Tracking in the Internet of Things," *IEEE Access*, DOI: 10.1109/ACCESS.2018.2875734, Oct. 2018.
 - [10] H. Noguchi, T. Demizu, N. Hoshikawa, M. Kataoka and Y. Yamato, "Autonomous Device Identification Architecture for Internet of Things," 2018 IEEE 4th World Forum on Internet of Things (WF-IoT 2018), pp.407-411, Feb. 2018.
 - [11] P. C. Evans and M. Annunziata, "Industrial Internet: Pushing the Boundaries of Minds and Machines," Technical report of General Electric (GE), Nov. 2012.
 - [12] T. Sterling, M. Anderson and M. Brodowicz, "High performance computing : modern systems and practices," Cambridge, MA : Morgan Kaufmann, ISBN 9780124202153, 2018.
 - [13] J. Sanders and E. Kandrot, "CUDA by example : an introduction to general-purpose GPU programming," Addison-Wesley, 2011.
 - [14] J. E. Stone, D. Gohara and G. Shi, "OpenCL: A parallel programming standard for heterogeneous computing systems," *Computing in science & engineering*, Vol.12, No.3, pp.66-73, 2010.
 - [15] Y. Yamato, "Automatic Offloading Method of Loop Statements of Software to FPGA," *International Journal of Parallel, Emergent and Distributed Systems*, Taylor and Francis, DOI: 10.1080/17445760.2021.1916020, Apr. 2021.
 - [16] Y. Yamato, "Proposal and evaluation of GPU offloading parts reconfiguration during applications operations for environment adaptation," *Journal of Network and Systems Management*, Springer, DOI: 10.1007/s10922-023-09789-2, Nov. 2023.
 - [17] Y. Yamato, "Study and Evaluation of Automatic Offloading Method in Mixed Offloading Destination Environment," *Cogent Engineering*, Taylor & Francis, Vol.9, Issue 1, DOI: 10.1080/23311916.2022.2080624, June 2022.
 - [18] Y. Yamato, "Study and Evaluation of Automatic GPU Offloading Method from Various Language Applications," *International Journal of Parallel, Emergent and Distributed Systems*, Taylor and Francis, DOI: 10.1080/17445760.2021.1971666, Sep. 2021.
 - [19] Y. Yamato, "Proposal of Automatic Offloading for Function Blocks of Applications," *The 8th IIAE International Conference on Industrial Application Engineering 2020 (ICIAE 2020)*, pp.4-11, Mar. 2020.
 - [20] Y. Yamato, "Improvement Proposal of Automatic GPU Offloading Technology," *The 8th International Conference on Information and Education Technology (ICIET 2020)*, pp.242-246, Mar. 2020.
 - [21] Y. Yamato, "Proposal of Automatic GPU Offloading Method from Various Language Applications," *The 9th International Conference on Information and Education Technology (ICIET 2021)*, pp.400-404, Mar. 2021.
 - [22] Y. Yamato, "Study and Evaluation of Improved Automatic GPU Offloading Method," *International Journal of Parallel, Emergent and Distributed Systems*, Taylor and Francis, DOI: 10.1080/17445760.2021.1941010, June 2021.
 - [23] Azure quantum website, <https://azure.microsoft.com/products/quantum/>
 - [24] J. Fung and M. Steve, "Computer vision signal processing on graphics processing units," 2004 IEEE International Conference on Acoustics, Speech, and Signal Processing, Vol. 5, pp.93-96, 2004.
 - [25] S. Wienke, P. Springer, C. Terboven and D. an Mey, "OpenACC-first experiences with real-world applications," *Euro-Par 2012 Parallel Processing*, pp.859-870, 2012.
 - [26] M. Wolfe, "Implementing the PGI accelerator model," *ACM the 3rd Workshop on General-Purpose Computation on Graphics Processing Units*, pp.43-50, Mar. 2010.
 - [27] E. Su, X. Tian, M. Girkar, G. Haab, S. Shah and P. Petersen, "Compiler support of the workqueueing execution model for Intel SMP architectures," *In Fourth European Workshop on OpenMP*, Sep. 2002.
 - [28] NVIDIA BlueField website, <https://www.nvidia.com/networking/products/data-processing-unit/>

- [29] Deckard website, <http://github.com/skyhover/Deckard>
- [30] Qiskit website, <https://www.ibm.com/quantum/qiskit>
- [31] Clang website, <http://llvm.org/>
- [32] Lark website, <https://lark-parser.readthedocs.io/en/stable/>
- [33] CloneDigger website, <https://pypi.org/project/clonedigger/>
- [34] Eigenvalue Solver website, <https://learning.quantum.ibm.com/course/variational-algorithm-design/instances-and-extensions>
- [35] J. Chen, B. Joo, W. Watson III and R. Edwards, "Automatic offloading C++ expression templates to CUDA enabled GPUs," 2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum, pp.2359-2368, May 2012.
- [36] C. Bertolli, S. F. Antao, G. T. Bercea, A. C. Jacob, A. E. Eichenberger, T. Chen, Z. Sura, H. Sung, G. Rokos, D. Appelhans and K. O'Brien, "Integrating GPU support for OpenMP offloading directives into Clang," ACM Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM'15), Nov. 2015.
- [37] S. Lee, S.J. Min and R. Eigenmann, "OpenMP to GPGPU: a compiler framework for automatic translation and optimization," 14th ACM SIGPLAN symposium on Principles and practice of parallel programming (PPoPP'09), 2009.
- [38] Cheng Liu, Ho-Cheung Ng and Hayden Kwok-Hay So, "Automatic nested loop acceleration on fpgas using soft CGRA overlay," Second International Workshop on FPGAs for Software Programmers (FSP 2015), 2015.
- [39] C. Alias, A. Darte and A. Plesco, "Optimizing remote accesses for offloaded kernels: Application to high-level synthesis for FPGA," 2013 Design, Automation and Test in Europe (DATE), pp.575-580, Mar. 2013.
- [40] L. Sommer, J. Korinth and A. Koch, "OpenMP device offloading to FPGA accelerators," 2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP 2017), pp.201-205, July 2017.
- [41] K. Prateek and M. Soumyadev, "Quantum programming on azure quantum - an open source tool for quantum developers," Quantum Computing: A Shift from Bits to Qubits, Springer, pp.283-309, 2023.
- [42] M. Mykhailova, "Teaching Quantum Computing using Microsoft Quantum Development Kit and Azure Quantum," 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), Vol.3, IEEE, 2023.
- [43] B. Michalowicz, K. Kandadi Suresh, H.Subramoni, D. Panda and S.Poole, "DPU-Bench: A Micro-Benchmark Suite to Measure Offload Efficiency Of SmartNICs," In Practice and Experience in Advanced Research Computing, pp.94-101, 2023.